

# Interstitial ads

[Page Summary](#)

**Select platform:** [Android \(beta\)](#) NEW (</admob/android/next-gen/interstitial>)

[Android](/admob/android/interstitial) (</admob/android/interstitial>)

[iOS](/admob/ios/interstitial) (</admob/ios/interstitial>)

[Unity](/admob/unity/interstitial) (</admob/unity/interstitial>)

[Flutter](/admob/flutter/interstitial) (</admob/flutter/interstitial>)



Interstitial ads are full-screen ads that cover the interface of their host app.

They're typically displayed at natural

transition points in the flow of an app, such as between activities or during the pause between levels in a game. When an app shows an interstitial ad, the user has the choice to either tap on the ad and continue to its destination or close it and return to the app. Read one of our [case studies](#)



([//admob.google.com/home/resources/freaking-math-powers-revenue-increase-with-google-admob-support/](https://admob.google.com/home/resources/freaking-math-powers-revenue-increase-with-google-admob-support/)).

This guide explains how to integrate interstitial ads into an Android app.

## Prerequisites

- Google Mobile Ads SDK 19.7.0 or higher.
- [Set up Google Mobile Ads SDK](/admob/android/quick-start) (</admob/android/quick-start>).

## Always test with test ads

When building and testing your apps, make sure you use test ads rather than live, production ads. Failure to do so can lead to suspension of your account.

The easiest way to load test ads is to use our dedicated test ad unit ID for Android interstitials:

**ca-app-pub-3940256099942544/1033173712**

It's been specially configured to return test ads for every request, and you're free to use it in your own apps while coding, testing, and debugging. Just make sure you replace it with your own ad unit ID before publishing your app.

For details on Google Mobile Ads SDK test ads, see [Enable test ads \(/admob/android/test-ads\)](/admob/android/test-ads).

## Load an ad

**Note:** Make all calls to the Mobile Ads SDK on the main thread.

To load an interstitial ad, call the `InterstitialAd` static `load()`.

(/admob/android/reference/com/google/android/gms/ads/interstitial/InterstitialAd#load(android.content.Context,java.lang.String,com.google.android.gms.ads.AdRequest,com.google.android.gms.ads.interstitial.InterstitialAdLoadCallback))

method and pass in an `InterstitialAdLoadCallback` to receive the loaded ad or any possible errors. Notice that like other format load callbacks, `InterstitialAdLoadCallback` leverages `LoadAdError` to provide higher fidelity error details.

**JavaKotlin** (#kotlin)  
(#java)

```
InterstitialAd.load(  
    this,  
    AD_UNIT_ID,  
    new AdRequest.Builder().build(),  
    new InterstitialAdLoadCallback() {  
        @Override  
        public void onAdLoaded(@NonNull InterstitialAd interstitialAd) {  
            Log.d(TAG, "Ad was loaded.");  
            MainActivity.this.interstitialAd = interstitialAd;  
        }  
  
        @Override  
        public void onAdFailedToLoad(@NonNull LoadAdError loadAdError) {  
            Log.d(TAG, loadAdError.getMessage());  
            interstitialAd = null;  
        }  
    });  
le/app/src/main/java/com/google/android/gms/example/interstitialexample/MyActivity.java#L136-L209
```

**Tip:** You can use ad load calls to build up a cache of preloaded ads before you intend to show them, so that ads can be shown with zero latency when needed. Since ads expire after an hour, you should clear this cache and reload with new ads every hour.

## Set the FullScreenContentCallback

The `FullScreenContentCallback` handles events related to displaying your `InterstitialAd`. Before showing `InterstitialAd`, make sure to set the callback:

JavaKotlin (#kotlin)  
(#java)

```
interstitialAd.setFullScreenContentCallback(
    new FullScreenContentCallback() {
        @Override
        public void onAdDismissedFullScreenContent() {
            // Called when fullscreen content is dismissed.
            Log.d(TAG, "The ad was dismissed.");
            // Make sure to set your reference to null so you don't
            // show it a second time.
            MainActivity.this.interstitialAd = null;
        }

        @Override
        public void onAdFailedToShowFullScreenContent(AdError adError) {
            // Called when fullscreen content failed to show.
            Log.d(TAG, "The ad failed to show.");
            // Make sure to set your reference to null so you don't
            // show it a second time.
            MainActivity.this.interstitialAd = null;
        }

        @Override
        public void onAdShowedFullScreenContent() {
            // Called when fullscreen content is shown.
            Log.d(TAG, "The ad was shown.");
        }

        @Override
        public void onAdImpression() {
            // Called when an impression is recorded for an ad.
            Log.d(TAG, "The ad recorded an impression.");
        }
    }
);
```

```

    }

    @Override
    public void onAdClicked() {
        // Called when ad is clicked.
        Log.d(TAG, "The ad was clicked.");
    }
});

```

[le/app/src/main/java/com/google/android/gms/example/interstitialexample/MyActivity.java#L149-L186](#)

## Show the ad

Interstitial ads should be displayed during natural pauses in the flow of an app. Between levels of a game is a good example, or after the user completes a task. To show an interstitial, use the [show\(.\)](#) ([/admob/android/reference/com/google/android/gms/ads/interstitial/InterstitialAd#show\(android.app.Activity\)](#)) method.

**JavaKotlin** (#kotlin)  
(#java)

```

if (interstitialAd != null) {
    interstitialAd.show(this);
} else {
    Log.d(TAG, "The interstitial ad is still loading.");
}

```

[le/app/src/main/java/com/google/android/gms/example/interstitialexample/MyActivity.java#L303-L313](#)

## Some best practices

### Consider whether interstitial ads are the right type of ad for your app.

Interstitial ads work best in apps with natural transition points. The conclusion of a task within an app, like sharing an image or completing a game level, creates such a point. Make sure you consider at which points in your app's workflow you'll display interstitial ads and how the user is likely to respond.

### Remember to pause the action when displaying an interstitial ad.

There are a number of different types of interstitial ads: text, image, video, and more. It's important to make sure that when your app displays an interstitial ad, it also suspends its use of some resources to allow the ad to take advantage of them. For example, when you make the call to display an interstitial ad, be sure to pause any audio output being produced by your app.

### Allow for adequate loading time.

Just as it's important to make sure you display interstitial ads at an appropriate time, it's also important to make sure the user doesn't have to wait for them to load. Loading the ad in advance by calling `load()`.

```
(/admob/android/reference/com/google/android/gms/ads/interstitial/InterstitialAd#load(android.content.Context,java.lang.String,com.google.android.gms.ads.AdRequest,com.google.android.gms.ads.interstitial.InterstitialAdLoadCallback))
```

before you intend to call `show()` can ensure that your app has a fully loaded interstitial ad at the ready when the time comes to display one.

### Don't flood the user with ads.

While increasing the frequency of interstitial ads in your app might seem like a great way to increase revenue, it can also degrade the user experience and lower clickthrough rates. Make sure that users aren't so frequently interrupted that they're no longer able to enjoy the use of your app.

## Source code

### + MyActivity.java

```
/*
 * Copyright (C) 2013 Google, Inc.
 *
 * Licensed under the Apache License, Version 2.0 (the "License");
 * you may not use this file except in compliance with the License.
 * You may obtain a copy of the License at
 *
 *     http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
```

```

*/
package com.google.android.gms.example.interstitialexample;

import android.annotation.SuppressLint;
import android.os.Bundle;
import android.os.CountDownTimer;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;
import com.google.android.gms.ads.AdError;
import com.google.android.gms.ads.AdRequest;
import com.google.android.gms.ads.FullScreenContentCallback;
import com.google.android.gms.ads.LoadAdError;
import com.google.android.gms.ads.MobileAds;
import com.google.android.gms.ads.RequestConfiguration;
import com.google.android.gms.ads.interstitial.InterstitialAd;
import com.google.android.gms.ads.interstitial.InterstitialAdLoadCallback;
import java.util.Arrays;
import java.util.concurrent.atomic.AtomicBoolean;

/** Main Activity. Inflates main activity xml. */
@SuppressLint("SetTextI18n")
public class MyActivity extends AppCompatActivity {

    // Check your logcat output for the test device hashed ID e.g.
    // "Use RequestConfiguration.Builder().setTestDeviceIds(Arrays.asList("ABCDEF012345
    // to get test ads on this device" or
    // "Use new ConsentDebugSettings.Builder().addTestDeviceHashedId("ABCDEF012345") to
    // a debug device".
    public static final String TEST_DEVICE_HASHED_ID = "ABCDEF012345";

    private static final long GAME_LENGTH_MILLISECONDS = 3000;
    private static final String AD_UNIT_ID = "ca-app-pub-3940256099942544/1033173712";
    private static final String TAG = "MyActivity";

    private final AtomicBoolean isMobileAdsInitializeCalled = new AtomicBoolean(false);
    private GoogleMobileAdsConsentManager googleMobileAdsConsentManager;
    private InterstitialAd interstitialAd;

```

```
private CountDownTimer countDownTimer;
private Button retryButton;
private boolean gamePaused;
private boolean gameOver;
private boolean adIsLoading;
private long timerMilliseconds;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_my);
    setSupportActionBar(findViewById(R.id.toolBar));

    ViewCompat.setOnApplyWindowInsetsListener(
        findViewById(R.id.container),
        (view, insets) -> {
            Insets systemBarsInsets = insets.getInsets(WindowInsetsCompat.Type.systemBa
            view.setPadding(
                systemBarsInsets.left, 0, systemBarsInsets.right, systemBarsInsets.bott
            return insets;
        });

    // Log the Mobile Ads SDK version.
    Log.d(TAG, "Google Mobile Ads SDK Version: " + MobileAds.getVersion());

    googleMobileAdsConsentManager =
        GoogleMobileAdsConsentManager.getInstance(getApplicationContext());
    googleMobileAdsConsentManager.gatherConsent(
        this,
        consentError -> {
            if (consentError != null) {
                // Consent not obtained in current session.
                Log.w(
                    TAG,
                    String.format("%s: %s", consentError.getErrorCode(), consentError.get
            }

            startGame();

            if (googleMobileAdsConsentManager.canRequestAds()) {
                initializeMobileAdsSdk();
            }

            if (googleMobileAdsConsentManager.isPrivacyOptionsRequired()) {
                // Regenerate the options menu to include a privacy setting.
                invalidateOptionsMenu();
            }
        });
}
```

```

// This sample attempts to load ads using consent obtained in the previous sessio
if (googleMobileAdsConsentManager.canRequestAds()) {
    initializeMobileAdsSdk();
}

// Create the "retry" button, which tries to show an interstitial between game pl
retryButton = findViewById(R.id.retry_button);
retryButton.setVisibility(View.INVISIBLE);
retryButton.setOnClickListener(
    new View.OnClickListener() {
        @Override
        public void onClick(View view) {
            showInterstitial();
        }
    });
}

public void loadAd() {
    // Request a new ad if one isn't already loaded.
    if (adIsLoading || interstitialAd != null) {
        return;
    }
    adIsLoading = true;
    InterstitialAd.load(
        this,
        AD_UNIT_ID,
        new AdRequest.Builder().build(),
        new InterstitialAdLoadCallback() {
            @Override
            public void onAdLoaded(@NonNull InterstitialAd interstitialAd) {
                Log.d(TAG, "Ad was loaded.");
                MyActivity.this.interstitialAd = interstitialAd;
                adIsLoading = false;
                Toast.makeText(MyActivity.this, "onAdLoaded()", Toast.LENGTH_SHORT).show(
                    interstitialAd.setFullScreenContentCallback(
                        new FullScreenContentCallback() {
                            @Override
                            public void onAdDismissedFullScreenContent() {
                                // Called when fullscreen content is dismissed.
                                Log.d(TAG, "The ad was dismissed.");
                                // Make sure to set your reference to null so you don't
                                // show it a second time.
                                MyActivity.this.interstitialAd = null;
                            }

                            @Override
                            public void onAdFailedToShowFullScreenContent(AdError adError) {

```



```

        // Called when fullscreen content failed to show.
        Log.d(TAG, "The ad failed to show.");
        // Make sure to set your reference to null so you don't
        // show it a second time.
        MainActivity.this.interstitialAd = null;
    }

    @Override
    public void onAdShowedFullScreenContent() {
        // Called when fullscreen content is shown.
        Log.d(TAG, "The ad was shown.");
    }

    @Override
    public void onAdImpression() {
        // Called when an impression is recorded for an ad.
        Log.d(TAG, "The ad recorded an impression.");
    }

    @Override
    public void onAdClicked() {
        // Called when ad is clicked.
        Log.d(TAG, "The ad was clicked.");
    }
});

}

@Override
public void onAdFailedToLoad(@NonNull LoadAdError loadAdError) {
    Log.d(TAG, loadAdError.getMessage());
    interstitialAd = null;
    adIsLoading = false;
    String error =
        String.format(
            java.util.Locale.US,
            "domain: %s, code: %d, message: %s",
            loadAdError.getDomain(),
            loadAdError.getCode(),
            loadAdError.getMessage());
    Toast.makeText(
        MainActivity.this, "onAdFailedToLoad() with error: " + error, Toast
        .show());
}
});
}

private void createTimer(final long milliseconds) {
    // Create the game timer, which counts down to the end of the level

```

```
// and shows the "retry" button.
if (countDownTimer != null) {
    countDownTimer.cancel();
}

final TextView textView = findViewById(R.id.timer);

countDownTimer =
    new CountDownTimer(milliseconds, 50) {
        @Override
        public void onTick(long millisUnitFinished) {
            timerMilliseconds = millisUnitFinished;
            textView.setText("seconds remaining: " + ((millisUnitFinished / 1000) + 1));
        }

        @Override
        public void onFinish() {
            gameOver = true;
            textView.setText("done!");
            retryButton.setVisibility(View.VISIBLE);
        }
    };

countDownTimer.start();
}

@Override
public void onResume() {
    // Start or resume the game.
    super.onResume();
    resumeGame();
}

@Override
public void onPause() {
    super.onPause();
    pauseGame();
}

@Override
public boolean onCreateOptionsMenu(Menu menu) {
    getMenuInflater().inflate(R.menu.action_menu, menu);
    return true;
}

@Override
public boolean onOptionsItemSelected(MenuItem item) {
    View menuItemView = findViewById(item.getItemId());
```

```

PopupMenu popup = new PopupMenu(this, menuItemView);
popup.getMenuInflater().inflate(R.menu.popup_menu, popup.getMenu());
popup.show();
popup
    .getMenu()
    .findItem(R.id.privacy_settings)
    .setVisible(googleMobileAdsConsentManager.isPrivacyOptionsRequired());
popup.setOnMenuItemClickListener(
    popupMenuItem -> {
        if (popupMenuItem.getItemId() == R.id.privacy_settings) {
            pauseGame();
            // Handle changes to user consent.
            googleMobileAdsConsentManager.showPrivacyOptionsForm(
                this,
                formError -> {
                    if (formError != null) {
                        Toast.makeText(this, formError.getMessage(), Toast.LENGTH_SHORT).
                    }
                    resumeGame();
                });
            return true;
        } else if (popupMenuItem.getItemId() == R.id.ad_inspector) {
            MobileAds.openAdInspector(
                this,
                error -> {
                    // Error will be non-null if ad inspector closed due to an error.
                    if (error != null) {
                        Toast.makeText(this, error.getMessage(), Toast.LENGTH_SHORT).show
                    }
                });
            return true;
        }
        return false;
    });
return super.onOptionsItemSelected(item);
}

private void showInterstitial() {
    // Show the ad if it's ready. Otherwise restart the game.
    if (interstitialAd != null) {
        interstitialAd.show(this);
    } else {
        Log.d(TAG, "The interstitial ad is still loading.");
        startGame();
        if (googleMobileAdsConsentManager.canRequestAds()) {
            loadAd();
        }
    }
}

```

```

}

private void startGame() {
    // Hide the button, and kick off the timer.
    retryButton.setVisibility(View.INVISIBLE);
    createTimer(GAME_LENGTH_MILLISECONDS);
    gamePaused = false;
    gameOver = false;
}

private void resumeGame() {
    if (gameOver || !gamePaused) {
        return;
    }
    // Create a new timer for the correct length.
    gamePaused = false;
    createTimer(timerMilliseconds);
}

private void pauseGame() {
    if (gameOver || gamePaused) {
        return;
    }
    countdownTimer.cancel();
    gamePaused = true;
}

private void initializeMobileAdsSdk() {
    if (isMobileAdsInitializeCalled.getAndSet(true)) {
        return;
    }

    // Set your test devices.
    MobileAds.setRequestConfiguration(
        new RequestConfiguration.Builder()
            .setTestDeviceIds(Arrays.asList(TEST_DEVICE_HASHED_ID))
            .build());

    new Thread(
        () -> {
            // Initialize the Google Mobile Ads SDK on a background thread.
            MobileAds.initialize(this, initializationStatus -> {});

            // Load an ad on the main thread.
            runOnUiThread(() -> loadAd());
        })
        .start();
}

```

```

    }
}

```

## MainActivity.kt

```

package com.google.android.gms.example.interstitialexample

import android.os.Bundle
import android.os.CountDownTimer
import android.util.Log
import android.view.Menu
import android.view.MenuItem
import android.view.View
import android.widget.PopupMenu
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import com.google.android.gms.ads.*
import com.google.android.gms.ads.interstitial.InterstitialAd
import com.google.android.gms.ads.interstitial.InterstitialAdLoadCallback
import com.google.android.gms.example.interstitialexample.databinding.ActivityMainBin
import java.util.concurrent.atomic.AtomicBoolean
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

class MainActivity : AppCompatActivity() {

    private val isMobileAdsInitializeCalled = AtomicBoolean(false)
    private lateinit var binding: ActivityMainBinding
    private lateinit var googleMobileAdsConsentManager: GoogleMobileAdsConsentManager
    private var interstitialAd: InterstitialAd? = null
    private var countdownTimer: CountDownTimer? = null
    private var gamePaused = false
    private var gameOver = false
    private var adIsLoading: Boolean = false
    private var timerMilliseconds = 0L

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        val view = binding.root
        setContentView(view)

```

```

setSupportActionBar(binding.toolbar)

ViewCompat.setOnApplyWindowInsetsListener(binding.root) { view, insets ->
    val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
    view.setPadding(systemBars.left, 0, systemBars.right, systemBars.bottom)
    insets
}

// Log the Mobile Ads SDK version.
Log.d(TAG, "Google Mobile Ads SDK Version: " + MobileAds.getVersion())

googleMobileAdsConsentManager = GoogleMobileAdsConsentManager.getInstance(this)
googleMobileAdsConsentManager.gatherConsent(this) { consentError ->
    if (consentError != null) {
        // Consent not obtained in current session.
        Log.w(TAG, "${consentError.errorCode}: ${consentError.message}")
    }

    // Kick off the first play of the "game".
    startGame()

    if (googleMobileAdsConsentManager.canRequestAds) {
        initializeMobileAdsSdk()
    }
    if (googleMobileAdsConsentManager.isPrivacyOptionsRequired) {
        // Regenerate the options menu to include a privacy setting.
        invalidateOptionsMenu()
    }
}

// This sample attempts to load ads using consent obtained in the previous session.
if (googleMobileAdsConsentManager.canRequestAds) {
    initializeMobileAdsSdk()
}

// Create the "retry" button, which triggers an interstitial between game plays.
binding.retryButton.visibility = View.INVISIBLE
binding.retryButton.setOnClickListener { showInterstitial() }
}

override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate(R.menu.action_menu, menu)
    return super.onCreateOptionsMenu(menu)
}

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    val menuItemView = findViewById<View>(item.itemId)
    val activity = this

```

```

PopupMenu(this, menuItemView).apply {
    menuInflater.inflate(R.menu.popup_menu, menu)
    menu
        .findItem(R.id.privacy_settings)
        .setVisible(googleMobileAdsConsentManager.isPrivacyOptionsRequired)
    show()
    setOnMenuItemClickListener { popupMenuItem ->
        when (popupMenuItem.itemId) {
            R.id.privacy_settings -> {
                pauseGame()
                // Handle changes to user consent.
                googleMobileAdsConsentManager.showPrivacyOptionsForm(activity) { formError
                    if (formError != null) {
                        Toast.makeText(activity, formError.message, Toast.LENGTH_SHORT).show(
                    )
                }
                resumeGame()
            }
            true
        }
        R.id.ad_inspector -> {
            MobileAds.openAdInspector(activity) { error ->
                // Error will be non-null if ad inspector closed due to an error.
                error?.let { Toast.makeText(activity, it.message, Toast.LENGTH_SHORT).s
            }
            true
        }
        // Handle other branches here.
        else -> false
    }
}
return super.onOptionsItemSelected(item)
}
}

private fun loadAd() {
    // Request a new ad if one isn't already loaded.
    if (adIsLoading || interstitialAd != null) {
        return
    }
    adIsLoading = true

    InterstitialAd.load(
        this,
        AD_UNIT_ID,
        AdRequest.Builder().build(),
        object : InterstitialAdLoadCallback() {
            override fun onAdLoaded(ad: InterstitialAd) {
                Log.d(TAG, "Ad was loaded.")
            }
        }
    )
}

```

```

        interstitialAd = ad
        adIsLoading = false
        Toast.makeText(this@MainActivity, "onAdLoaded()", Toast.LENGTH_SHORT).show()
    }

    override fun onAdFailedToLoad(adError: LoadAdError) {
        Log.d(TAG, adError.message)
        interstitialAd = null
        adIsLoading = false
        val error =
            "domain: ${adError.domain}, code: ${adError.code}, " + "message: ${adError.message}"
        Toast.makeText(
            this@MainActivity,
            "onAdFailedToLoad() with error $error",
            Toast.LENGTH_SHORT,
        )
            .show()
    }
},
)
}

// Create the game timer, which counts down to the end of the level
// and shows the "retry" button.
private fun createTimer(milliseconds: Long) {
    countdownTimer?.cancel()

    countdownTimer =
        object : CountDownTimer(milliseconds, 50) {
            override fun onTick(millisUntilFinished: Long) {
                timerMilliseconds = millisUntilFinished
                binding.timer.text = "seconds remaining: ${ millisUntilFinished / 1000 + 1 }"
            }

            override fun onFinish() {
                gameOver = true
                binding.timer.text = "done!"
                binding.retryButton.visibility = View.VISIBLE
            }
        }

    countdownTimer?.start()
}

// Show the ad if it's ready. Otherwise restart the game.
private fun showInterstitial() {
    if (interstitialAd != null) {
        interstitialAd?.fullScreenContentCallback =

```



```

object : FullScreenContentCallback() {
    override fun onAdDismissedFullScreenContent() {
        // Called when fullscreen content is dismissed.
        Log.d(TAG, "Ad was dismissed.")
        // Don't forget to set the ad reference to null so you
        // don't show the ad a second time.
        interstitialAd = null
    }

    override fun onAdFailedToShowFullScreenContent(adError: AdError) {
        // Called when fullscreen content failed to show.
        Log.d(TAG, "Ad failed to show.")
        // Don't forget to set the ad reference to null so you
        // don't show the ad a second time.
        interstitialAd = null
    }

    override fun onAdShowedFullScreenContent() {
        // Called when fullscreen content is shown.
        Log.d(TAG, "Ad showed fullscreen content.")
    }

    override fun onAdImpression() {
        // Called when an impression is recorded for an ad.
        Log.d(TAG, "Ad recorded an impression.")
    }

    override fun onAdClicked() {
        // Called when ad is clicked.
        Log.d(TAG, "Ad was clicked.")
    }
}

interstitialAd?.show(this)
} else {
    startGame()
    if (googleMobileAdsConsentManager.canRequestAds) {
        loadAd()
    }
}
}

// Hide the button, and kick off the timer.
private fun startGame() {
    binding.retryButton.visibility = View.INVISIBLE
    createTimer(GAME_LENGTH_MILLISECONDS)
    gamePaused = false
    gameOver = false
}

```

```

    }

    private fun pauseGame() {
        if (gameOver || gamePaused) {
            return
        }
        countdownTimer?.cancel()
        gamePaused = true
    }

    private fun resumeGame() {
        if (gameOver || !gamePaused) {
            return
        }
        createTimer(timerMilliseconds)
        gamePaused = true
    }

    private fun initializeMobileAdsSdk() {
        if (isMobileAdsInitializeCalled.getAndSet(true)) {
            return
        }

        // Set your test devices.
        MobileAds.setRequestConfiguration(
            RequestConfiguration.Builder().setTestDeviceIds(listOf(TEST_DEVICE_HASHED_ID)).
        )

        CoroutineScope(Dispatchers.IO).launch {
            // Initialize the Google Mobile Ads SDK on a background thread.
            MobileAds.initialize(this@MainActivity) {}
            runOnUiThread {
                // Load an ad on the main thread.
                loadAd()
            }
        }
    }

    // Resume the game if it's in progress.
    public override fun onResume() {
        super.onResume()
        resumeGame()
    }

    public override fun onPause() {
        super.onPause()
        pauseGame()
    }

```

```
companion object {
    // This is an ad unit ID for a test ad. Replace with your own interstitial ad unit ID
    private const val AD_UNIT_ID = "ca-app-pub-3940256099942544/1033173712"
    private const val GAME_LENGTH_MILLISECONDS = 3000L
    private const val TAG = "MainActivity"

    // Check your logcat output for the test device hashed ID e.g.
    // "Use RequestConfiguration.Builder().setTestDeviceIds(Arrays.asList("ABCDEF012345"))
    // to get test ads on this device" or
    // "Use new ConsentDebugSettings.Builder().addTestDeviceHashedId("ABCDEF012345")
    // a debug device".
    const val TEST_DEVICE_HASHED_ID = "ABCDEF012345"
}
}
```

## Examples on GitHub

- Interstitial ads example: [Java](#)  
([//github.com/googleads/googleads-mobile-android-examples/tree/main/java/admob/InterstitialExample](https://github.com/googleads/googleads-mobile-android-examples/tree/main/java/admob/InterstitialExample))  
| [Kotlin](#)  
([//github.com/googleads/googleads-mobile-android-examples/tree/main/kotlin/admob/InterstitialExample](https://github.com/googleads/googleads-mobile-android-examples/tree/main/kotlin/admob/InterstitialExample))

## Success stories

- [Sample use case](#)  
([//admob.google.com/home/resources/freaking-math-powers-revenue-increase-with-google-admob-support/](https://admob.google.com/home/resources/freaking-math-powers-revenue-increase-with-google-admob-support/))

## Next steps

- If you haven't already, create your own interstitial ad unit in the [AdMob web interface](#) ([//apps.admob.com](https://apps.admob.com)).
- Learn more about [ad targeting](#) ([/admob/android/targeting](https://admob.google.com/android/targeting)) and [interstitial ad guidance](#) ([//support.google.com/admob/answer/6066980](https://support.google.com/admob/answer/6066980)).

- Learn more about [user privacy](https://developers.google.com/admob/android/interstitial/privacy) (https://developers.google.com/admob/android/interstitial/privacy).
- Explore [optimized SDK initialization and ad loading \(beta\)](/admob/android/optimize-initialization) (/admob/android/optimize-initialization).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-02-27 UTC.